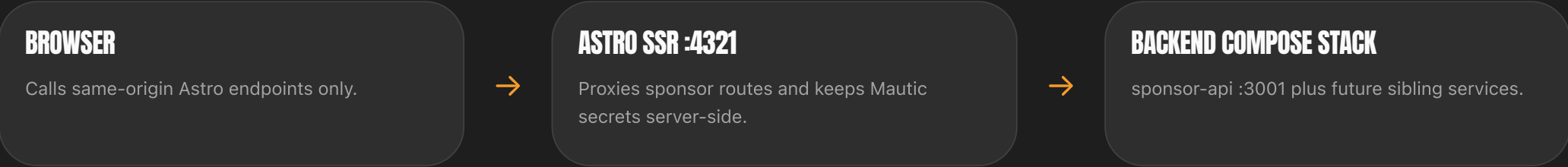


Coolify Split Deployment Connectors

Deploy the Astro site and backend tools as two Coolify apps. The frontend remains a single SSR container, while backend mini-servers live in one compose stack and are connected through server-side Astro proxy routes.



Frontend app

Recommended

Use the repository root Dockerfile.

Coolify should expose port 4321 and use /api/health for health checks.
Dockerfile.nginx is not needed because Coolify supplies the reverse proxy.

Build arg: `SITE_URL`

Runtime: `MAUTIC_*` , `BACKEND_SPONSOR_API_URL`

Backend app

Backend stack

Use services/docker-compose.yml.

The first service is sponsor-api on port 3001 with a named sponsor-data volume. Add future tools as sibling compose services.

Runtime: `ADMIN_SECRET` , `ALLOWED_ORIGIN` , `SPONSOR_API_PORT`

STEP-BY-STEP DEPLOYMENT

1. Deploy backend stack

Create a Coolify Docker Compose app using services/docker-compose.yml. Select sponsor-api as the public service on internal port 3001.
`services/docker-compose.yml`

2. Set backend secrets

Set ADMIN_SECRET, ALLOWED_ORIGIN, and SPONSOR_API_PORT. Assign a domain to sponsor-api after it deploys.
`ADMIN_SECRET=...`

3. Deploy frontend

Create a separate Coolify Dockerfile app from the repository root. Expose port 4321 and set the health check path to /api/health.
`Dockerfile`

4. Wire frontend

Set BACKEND_SPONSOR_API_URL to the backend Coolify domain. Keep Mautic variables on the frontend app only.
`BACKEND_SPONSOR_API_URL=...`

5. Verify

Open the frontend health endpoint. The sponsor backend should report ok once the URL is configured and reachable.
`curl /api/health`

APP SPLIT

App	Coolify type	Purpose	Operational note
Frontend	Dockerfile	Astro SSR on port 4321	Coolify proxy handles TLS and domains
Backend tools	Docker Compose	sponsor-api now, future mini-servers later	One Coolify stack with shared volumes

ENVIRONMENT VARIABLES

HEALTH CHECKS

/api/health

The frontend stays healthy even if the backend is not ready. This makes staged deploys safe while still surfacing connector status.

Expected once wired: `backend.sponsor: "ok"`

Before wiring: `backend.sponsor: "unconfigured"`

SAME-ORIGIN API ROUTES

- GET /api/sponsoring/catalog
- POST /api/sponsoring/verify-code
- POST /api/sponsoring/store-email

Variable	Where	Example	Why
SITE_URL	Frontend build	https://2026.synbioreactor.de	Build-time sitemap and robots URL
MAUTIC_BASE_URL	Frontend runtime	secret/config	Newsletter upstream base URL
MAUTIC_AUTH_TOKEN	Frontend runtime	secret	Bearer token for Mautic
BACKEND_SPONSOR_API_URL	Frontend runtime	backend URL	Server-side proxy target
ADMIN_SECRET	Backend runtime	secret	Sponsor admin API access
ALLOWED_ORIGIN	Backend runtime	frontend URL	CORS allowlist for direct backend calls
SPONSOR_API_PORT	Backend runtime	3001	Published sponsor API port

LOCAL VERIFICATION COMMANDS

Frontend compose config

docker compose config

Backend compose config

docker compose --env-file services/.env.example -f services/docker-compose.yml config

Astro build

bun run build